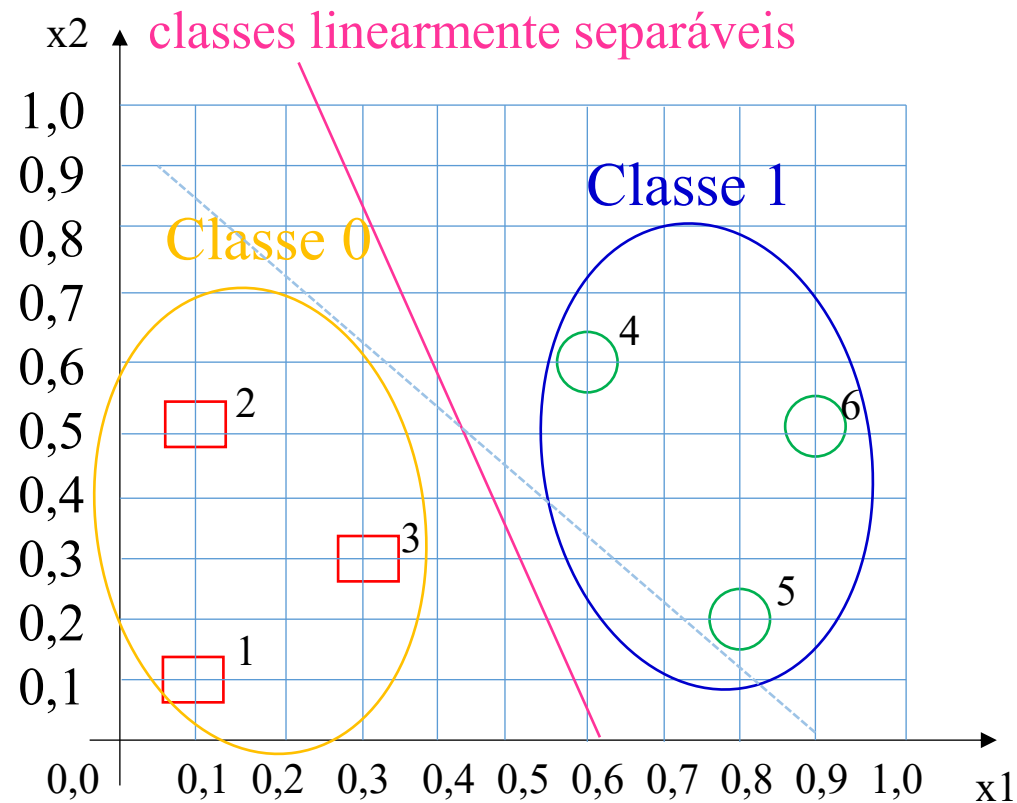




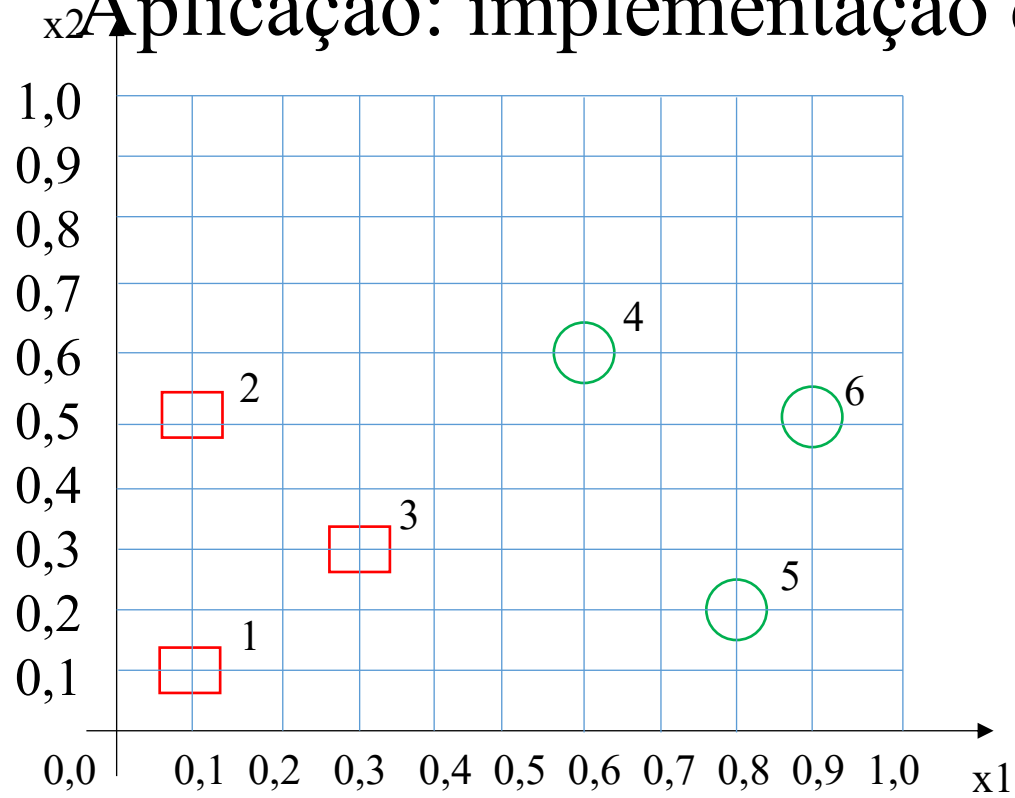
Redes Neurais Artificiais Aplicação com *Perceptron*

profº Mauricio Conceição Mario

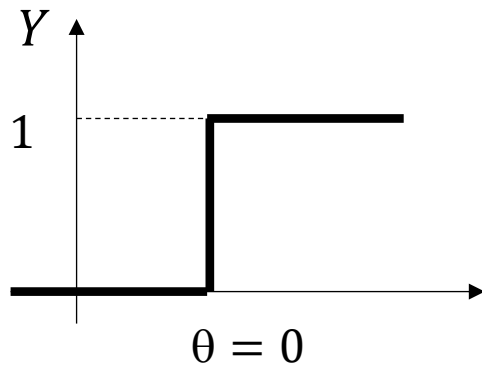
Aplicação: implementação de *Perceptron* para classificar 2 classes de números.



Aplicação: implementação de Perceptron



elemento	x1	x2	
1	0,1	0,1	Classe 0
2	0,1	0,5	
3	0,3	0,3	
4	0,6	0,6	Classe 1
5	0,8	0,2	
6	0,9	0,5	



Função de ativação de limiar

Uso do Perceptron para classificar padrões ou classes

Classe 0

$w_1 \ w_2$

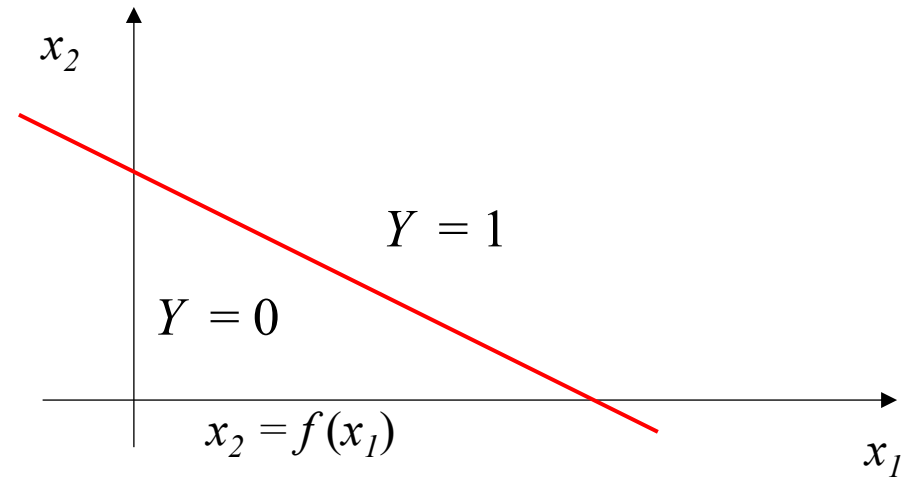
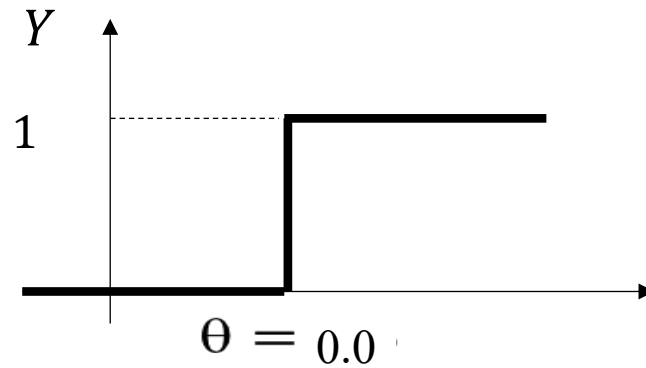
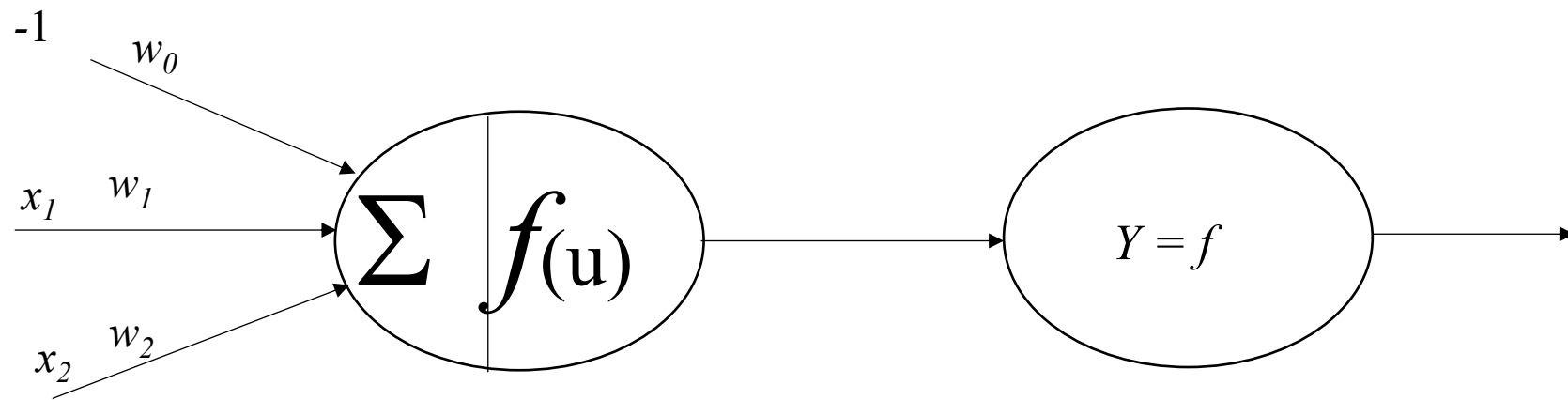
$\{0.1, 0.1\},$
 $\{0.1, 0.5\},$
 $\{0.3, 0.3\} \rightarrow Y = 0$

Classe 1

$w_1 \ w_2$

$\{0.6, 0.6\},$
 $\{0.8, 0.2\},$
 $\{0.9, 0.5\} \rightarrow Y = 1$

modelo da rede:



```
7
8 from __future__ import division
9 from collections import Counter
10 from functools import partial
11 from linear_algebra import dot
12 import matplotlib
13 import matplotlib.pyplot as plt
14
15 def degrau(x):
16     return 1 if x >= 0 else 0
17
18 def saida_perceptron(pesos, entradas):
19     y = dot(pesos, entradas)
20     return degrau(y)
21
22 def ajustes(sinapses, entradas, saida):
23
24     taxa_aprendizagem = 0.08
25
26     saida_parcial = saida_perceptron(sinapses, entradas)
27
28     for j in range(3):
29         sinapses[j] = sinapses[j] + taxa_aprendizagem * (saida[0] - saida_parcial) * entradas[j]
30
31     saida = saida_parcial
32     return sinapses, saida
33
34 def teste_generalizacao(sinapses, entradas, saida):
35     saida_parcial = saida_perceptron(sinapses, entradas)
36     saida = saida_parcial
37     return sinapses, saida
38
39 neuronio = [0.22, -0.33, 0.44] # pesos das sinapses do neurônio
40 padrao_0_0 = [-1, 0.1, 0.1]
41 padrao_0_1 = [-1, 0.1, 0.5]
42 padrao_0_2 = [-1, 0.3, 0.3]
43 padrao_1_0 = [-1, 0.6, 0.6]
44 padrao_1_1 = [-1, 0.8, 0.2]
45 padrao_1_2 = [-1, 0.9, 0.5]
46
47 saida0 = [0]
48 saida1 = [1]
```

View Help

Panels

☐ Lock panes and toolbars	Ctrl+Shift+F5	☐ Editor	
☒ Close current pane	Ctrl+Shift+F4	☐ IPython console	Ctrl+Shift+I
☒ Maximize current pane	Ctrl+Alt+Shift+M	☐ Variable explorer	Ctrl+Shift+V
Toolbars		☐ Help	Ctrl+Shift+H
Hide toolbars	Alt+Shift+T	☒ Plots	Ctrl+Shift+G
Window layouts		☐ Files	Ctrl+Shift+X
Use previous layout	Alt+Shift+PgUp	☐ Outline	Ctrl+Shift+O
Use next layout	Alt+Shift+PgDown	☐ Project	Ctrl+Shift+P
☒ Fullscreen mode	F11	☐ Find	Ctrl+Shift+F
☐ Attached console window (debugging)		☐ History	Ctrl+Shift+L
		☐ Profiler	
		☐ Breakpoints	Ctrl+Shift+B
		☐ Code Analysis	
		☐ Online help	Ctrl+Shift+D
		☐ Internal console	

Console 1/A x

```
In [17]: runfile('C:/Users/cmari/.spyder-py3/aplicação_perceptron.py', wdir='C:/Users/cmari/.spyder-py3')
[0.22, -0.33, 0.44] saida0 = 0
[0.22, -0.33, 0.44] saida0 = 0
[0.22, -0.33, 0.44] saida0 = 0
[0.14, -0.28200000000000003, 0.488] saida1 = 0
```

Python 3.8

File Edit Search Source Run Debug Consoles Projects Tools View Help

backpropagation_2.py backpropagation_4.py aplicação_perceptron.py grafico_4.py aproximação_funcional.py senoide.py adaline.py

C:\Users\cmari\.spyder-py3\aplicação_perceptron.py

```
40 padrao_0_0 = [-1, 0.1, 0.1]
41 padrao_0_1 = [-1, 0.1, 0.5]
42 padrao_0_2 = [-1, 0.3, 0.3]
43 padrao_1_0 = [-1, 0.6, 0.6]
44 padrao_1_1 = [-1, 0.8, 0.2]
45 padrao_1_2 = [-1, 0.9, 0.5]
46
47 saida0 = [0]
48 saida1 = [1]
49 # 11 ciclos de treinamento
50 n = 0;
51 for _ in range(11):
52     neuronio, saida_0 = ajustes(neuronio, padrao_0_0, saida0)
53     print(neuronio,"saida0 =", saida_0)
54     neuronio, saida_0 = ajustes(neuronio, padrao_0_1, saida0)
55     print(neuronio,"saida0 =", saida_0)
56     neuronio, saida_0 = ajustes(neuronio, padrao_0_2, saida0)
57     print(neuronio,"saida0 =", saida_0)
58     neuronio, saida_1 = ajustes(neuronio, padrao_1_0, saida1)
59     print(neuronio,"saida1 =", saida_1)
60     neuronio, saida_1 = ajustes(neuronio, padrao_1_1, saida1)
61     print(neuronio,"saida1 =", saida_1)
62     neuronio, saida_1 = ajustes(neuronio, padrao_1_2, saida1)
63     print(neuronio,"saida1 =", saida_1)
64     n = n + 1;
65     print("número de ciclos = ", n)
66
67
68 x = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
69 y = [-0.125, -0.0125, 0.1, 0.2125, 0.325, 0.4375, 0.55, 0.6625, 0.775, 0.8875]
70 # y = 1.0 - 1.125*x - equação da reta de separação de classes
71 x1 = [0.1, 0.1, 0.3]
72 x2 = [0.6, 0.8, 0.9]
73 y1 = [0.1, 0.5, 0.3]
74 y2 = [0.6, 0.2, 0.5]
75 #insere os pontos x1, y1, x2, y2 no gráfico
76 plt.scatter(x1, y1)
77 plt.scatter(x2, y2)
78
79 plt.plot(y, x, color = 'green', marker = '*', linestyle = '--')
80 #definição dos nomes das opções linestyle
81 """
```

Variable explorer Help Plots

Console 1/A

```
In [17]: runfile('C:/Users/cmari/.spyder-py3/aplicação_perceptron.py', wdir='C:/Users/cmari/.spyder-py3')
[0.22, -0.33, 0.44] saida0 = 0
[0.22, -0.33, 0.44] saida0 = 0
[0.22, -0.33, 0.44] saida0 = 0
[0.14, -0.2820000000000003, 0.488] saida1 = 0
[0.06000000000000001, -0.2180000000000003, 0.504] saida1 = 0
[-0.01999999999999999, -0.1460000000000002, 0.544] saida1 = 0
número de ciclos = 1
[0.06000000000000001, -0.1540000000000003, 0.536] saida0 = 1
[0.14, -0.1620000000000003, 0.4960000000000005] saida0 = 1
[0.14, -0.1620000000000003, 0.4960000000000005] saida0 = 0
[0.14, -0.1620000000000003, 0.4960000000000005] saida1 = 1
[0.06000000000000001, -0.0980000000000003, 0.512] saida1 = 0
[0.06000000000000001, -0.0980000000000003, 0.512] saida1 = 1
número de ciclos = 2
[0.06000000000000001, -0.0980000000000003, 0.512] saida0 = 0
[0.14, -0.1060000000000004, 0.4720000000000003] saida0 = 1
[0.14, -0.1060000000000004, 0.4720000000000003] saida0 = 0
[0.14, -0.1060000000000004, 0.4720000000000003] saida1 = 1
[0.06000000000000001, -0.0420000000000004, 0.4880000000000004] saida1 = 0
[0.06000000000000001, -0.0420000000000004, 0.4880000000000004] saida1 = 1
número de ciclos = 3
[0.06000000000000001, -0.0420000000000004, 0.4880000000000004] saida0 = 0
[0.14, -0.0500000000000004, 0.4480000000000006] saida0 = 1
[0.14, -0.0500000000000004, 0.4480000000000006] saida0 = 0
[0.14, -0.0500000000000004, 0.4480000000000006] saida1 = 1
[0.06000000000000001, 0.01399999999999964, 0.4640000000000001] saida1 = 0
[0.06000000000000001, 0.01399999999999964, 0.4640000000000001] saida1 = 1
número de ciclos = 4
[0.06000000000000001, 0.01399999999999964, 0.4640000000000001] saida0 = 0
[0.14, 0.00599999999999964, 0.4240000000000001] saida0 = 1
[0.14, 0.00599999999999964, 0.4240000000000001] saida0 = 0
[0.14, 0.00599999999999964, 0.4240000000000001] saida1 = 1
[0.06000000000000001, 0.0699999999999997, 0.4400000000000001] saida1 = 0
[0.06000000000000001, 0.0699999999999997, 0.4400000000000001] saida1 = 1
número de ciclos = 5
```

LSP Python: ready Kite: ready conda: base (Python 3.8.3) Line 64, Col 13 UTF-8 CRLF RW Mem 66%

Digite aqui para pesquisar

16:03 13/03/2021

FileEditSearchSourceRunDebugConsolesProjectsToolsViewHelp

C:\Users\cmari\.spyder-py3

backpropagation_2.pybackpropagation_4.pyaplicação_perceptron.pygrafico_4.pyaproximação_funcional.py senoide.pyadaline.py

```
40 padrao_0_0 = [-1, 0.1, 0.1]
41 padrao_0_1 = [-1, 0.1, 0.5]
42 padrao_0_2 = [-1, 0.3, 0.3]
43 padrao_1_0 = [-1, 0.6, 0.6]
44 padrao_1_1 = [-1, 0.8, 0.2]
45 padrao_1_2 = [-1, 0.9, 0.5]
46
47 saida0 = [0]
48 saida1 = [1]
49 # 11 ciclos de treinamento
50 n = 0;
51 for _ in range(11):
52     neuronio, saida_0 = ajustes(neuronio, padrao_0_0, saida0)
53     print (neuronio,"saida0 =", saida_0)
54     neuronio, saida_0 = ajustes(neuronio, padrao_0_1, saida0)
55     print (neuronio,"saida0 =", saida_0)
56     neuronio, saida_0 = ajustes(neuronio, padrao_0_2, saida0)
57     print (neuronio,"saida0 =", saida_0)
58     neuronio, saida_1 = ajustes(neuronio, padrao_1_0, saida1)
59     print (neuronio,"saida1 =", saida_1)
60     neuronio, saida_1 = ajustes(neuronio, padrao_1_1, saida1)
61     print (neuronio,"saida1 =", saida_1)
62     neuronio, saida_1 = ajustes(neuronio, padrao_1_2, saida1)
63     print (neuronio,"saida1 =", saida_1)
64     n = n + 1;
65     print ("número de ciclos = ", n)
66
67
68 x = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
69 y = [-0.125, -0.0125, 0.1, 0.2125, 0.325, 0.4375, 0.55, 0.6625, 0.775, 0.8875]
70 # y = 1.0 - 1.125*x - equação da reta de separação de classes
71 x1 = [0.1, 0.1, 0.3]
72 x2 = [0.6, 0.8, 0.9]
73 y1 = [0.1, 0.5, 0.3]
74 y2 = [0.6, 0.2, 0.5]
75 #insere os pontos x1, y1, x2, y2 no gráfico
76 plt.scatter(x1, y1)
77 plt.scatter(x2, y2)
78 #plota a reta y, x
79 plt.plot(y, x, color = 'green', marker = '*', linestyle = '--')
80 #definição dos nomes das opções linestyle
81 """
```

Variable explorerHelpPlots

Console 1/A

número de ciclos = 5
[0.06000000000000001, 0.06999999999999997, 0.4400000000000001] saida0 = 0
[0.14, 0.061999999999999965, 0.40000000000000013] saida0 = 1
[0.14, 0.061999999999999965, 0.40000000000000013] saida0 = 0
[0.14, 0.061999999999999965, 0.40000000000000013] saida1 = 1
[0.06000000000000001, 0.12599999999999997, 0.41600000000000015] saida1 = 0
[0.06000000000000001, 0.12599999999999997, 0.41600000000000015] saida1 = 1
número de ciclos = 6
[0.06000000000000001, 0.12599999999999997, 0.41600000000000015] saida0 = 0
[0.14, 0.11799999999999997, 0.37600000000000017] saida0 = 1
[0.22000000000000003, 0.09399999999999997, 0.35200000000000015] saida0 = 1
[0.22000000000000003, 0.09399999999999997, 0.35200000000000015] saida1 = 1
[0.14, 0.15799999999999997, 0.36800000000000016] saida1 = 0
[0.14, 0.15799999999999997, 0.36800000000000016] saida1 = 1
número de ciclos = 7
[0.14, 0.15799999999999997, 0.36800000000000016] saida0 = 0
[0.22000000000000003, 0.14999999999999997, 0.32800000000000002] saida0 = 1
[0.22000000000000003, 0.14999999999999997, 0.32800000000000002] saida0 = 0
[0.22000000000000003, 0.14999999999999997, 0.32800000000000002] saida1 = 1
[0.14, 0.21399999999999997, 0.34400000000000002] saida1 = 0
[0.14, 0.21399999999999997, 0.34400000000000002] saida1 = 1
número de ciclos = 8
[0.14, 0.21399999999999997, 0.34400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
número de ciclos = 9
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
número de ciclos = 10
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0

LSP Python: readyKite: readyconda: base (Python 3.8.3)Line 64, Col 13UTF-8CRLFRWMem 66%

Digite aqui para pesquisar

16:0513/03/2021

Perceptron – resultados no treinamento

C:\Users\cmari\.spyder-py3

C:\Users\cmari\.spyder-py3\aplicação_perceptron.py

backpropagation_2.py x backpropagation_4.py x aplicação_perceptron.py x grafico_4.py x aproximação_funcional.py x senoide.py x adaline.py

```

77 plt.scatter(x2, y2)
78 #plota a reta y, x
79 plt.plot(y, x, color = 'green', marker = '*', linestyle = '--')
80 #definição dos nomes das opções linestyle
81 """
82 a valid value for ls; supported values are '-',
83 '--', '-.', ':', 'None', ' ', ' ', 'solid', 'dashed', 'dashdot', 'dotted'
84 """
85 plt.title("separação de classes com Perceptron")
86
87 plt.xlabel("eixo x")
88 plt.ylabel("eixo y")
89 plt.show()

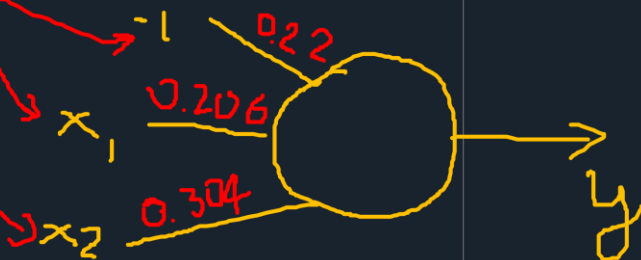
```

```

94 #teste de generalização
95 padrao_teste_0 = [-1, 0.2, 0.4]
96 padrao_teste_1 = [-1, 0.7, 0.8]
97 padrao_teste_2 = [-1, 0.6, 0.3]
98 padrao_teste_3 = [-1, 0.1, 0.9]
99 padrao_teste_4 = [-1, 0.2, 0.6]
100 padrao_teste_5 = [-1, 0.8, 0.1]
101 #teste de generalização
102 print("teste de generalização")
103 neuronio, saida_0 = teste_generalização(neuronio, padrao_teste_0, saida0)
104 print (neuronio,"saida0 =", saida_0)
105 neuronio, saida_1 = teste_generalização(neuronio, padrao_teste_1, saida1)
106 print (neuronio,"saida1 =", saida_1)
107 neuronio, saida_0 = teste_generalização(neuronio, padrao_teste_2, saida0)
108 print (neuronio,"saida0 =", saida_0)
109 neuronio, saida_1 = teste_generalização(neuronio, padrao_teste_3, saida1)
110 print (neuronio,"saida1 =", saida_1)
111 neuronio, saida_0 = teste_generalização(neuronio, padrao_teste_4, saida0)
112 print (neuronio,"saida0 =", saida_0)
113 neuronio, saida_1 = teste_generalização(neuronio, padrao_teste_5, saida1)
114 print (neuronio,"saida1 =", saida_1)
115
116
117

```

convergência dos valores dos pesos
para que saída = saída desejada



Console 1/A x

```

numero de ciclos = 0
[0.14, 0.21399999999999997, 0.34400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
numero de ciclos = 9
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
numero de ciclos = 10
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
numero de ciclos = 11
teste de generalização
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 0

```

In [18]: **Perceptron – resultado da generalização**

IPython console History

LSP Python: ready

Kite: ready

conda: base (Python 3.8.3)

Line 64, Col 13

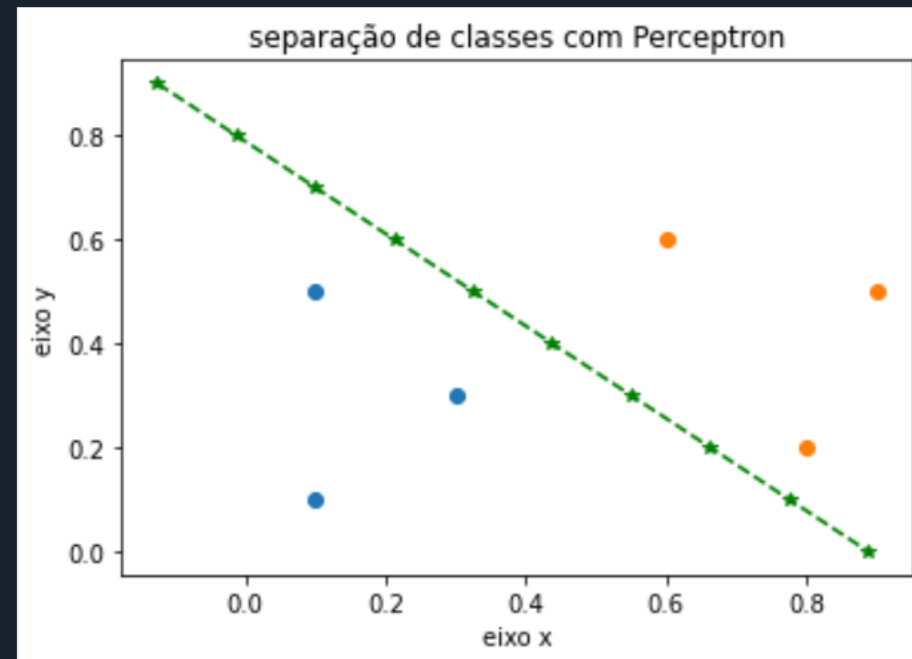
UTF-8

CRLF

RW

Mem 64%

```
49 # 11 ciclos de treinamento
50 n = 0;
51 for _ in range(11):
52     neuronio, saida_0 = ajustes(neuronio, padrao_0_0, saida0)
53     print(neuronio, "saida0 =", saida_0)
54     neuronio, saida_0 = ajustes(neuronio, padrao_0_1, saida0)
55     print(neuronio, "saida0 =", saida_0)
56     neuronio, saida_0 = ajustes(neuronio, padrao_0_2, saida0)
57     print(neuronio, "saida0 =", saida_0)
58     neuronio, saida_1 = ajustes(neuronio, padrao_1_0, saida1)
59     print(neuronio, "saida1 =", saida_1)
60     neuronio, saida_1 = ajustes(neuronio, padrao_1_1, saida1)
61     print(neuronio, "saida1 =", saida_1)
62     neuronio, saida_1 = ajustes(neuronio, padrao_1_2, saida1)
63     print(neuronio, "saida1 =", saida_1)
64     n = n + 1;
65     print("número de ciclos = ", n)
66
67
68 x = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1, 0.0]
69 y = [-0.125, -0.0125, 0.1, 0.2125, 0.325, 0.4375, 0.55, 0.6625, 0.775, 0.8875]
70 # y = 1.0 - 1.125*x - equação da reta de separação de classes
71 x1 = [0.1, 0.1, 0.3]
72 x2 = [0.6, 0.8, 0.9]
73 y1 = [0.1, 0.5, 0.3]
74 y2 = [0.6, 0.2, 0.5]
75 #insere os pontos x1, y1, x2, y2 no gráfico
76 plt.scatter(x1, y1)
77 plt.scatter(x2, y2)
78 #plota a reta y, x
79 plt.plot(y, x, color = 'green', marker = '*', linestyle = '--')
80 #definição dos nomes das opções linestyle
81 """
82 a valid value for ls; supported values are '-',
83 '--', '-.', ':', 'None', ' ', '', 'solid', 'dashed', 'dashdot', 'dotted'
84 """
85 plt.title("separação de classes com Perceptron")
86
87 plt.xlabel("eixo x")
88 plt.ylabel("eixo y")
89 plt.show()
90
```



Variable explorer Help Plots

Console 1/A x

```
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
número de ciclos = 11
teste de generalização
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 1
[0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 0
```

In [3]:

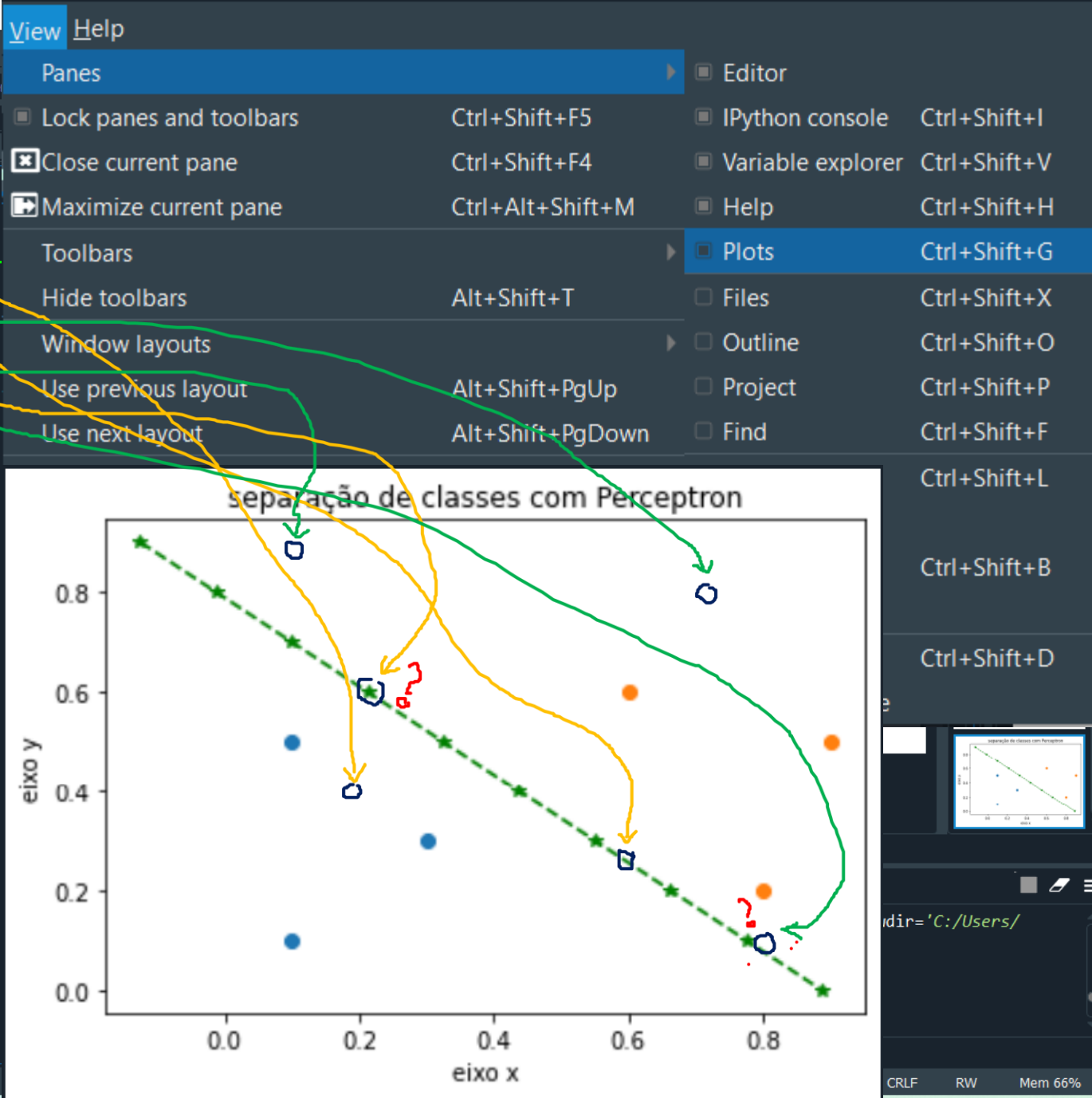
Spyder (Python 3.8)

File Edit Search Source Run Debug Consoles Projects Tools View Help

C:\Users\cmari\spyder-py3\aplicação_perceptron.py

backpropagation_2.py x backpropagation_4.py x aplicação_perceptron.py x grafico_4.py x aproximação_funcional.py x senoide.py

```
77 plt.scatter(y2, x2)
78 #plota a reta y, x
79 plt.plot(y, x, color = 'green', marker = '*', linestyle = '--')
80 #definição dos nomes das opções linestyle
81
82 teste de generalização
83 [0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
84 [0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
85 [0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 0
86 [0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 1
87 [0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida0 = 1
88 [0.22000000000000003, 0.20599999999999996, 0.30400000000000002] saida1 = 0
89
90 In [18]: Perceptron – resultado da generalização
91
92
93
94 #teste de generalização
95 padrao_teste_0 = [-1, 0.2, 0.4]
96 padrao_teste_1 = [-1, 0.7, 0.8]
97 padrao_teste_2 = [-1, 0.6, 0.3]
98 padrao_teste_3 = [-1, 0.1, 0.9]
99 padrao_teste_4 = [-1, 0.2, 0.6]
100 padrao_teste_5 = [-1, 0.8, 0.1]
101 #teste de generalização
102 print("teste de generalização")
103 neuronio, saida_0 = teste_generalização(neuronio, padrao_teste_0, saida0)
104 print(neuronio,"saida0 =", saida_0)
105 neuronio, saida_1 = teste_generalização(neuronio, padrao_teste_1, saida1)
106 print(neuronio,"saida1 =", saida_1)
107 neuronio, saida_0 = teste_generalização(neuronio, padrao_teste_2, saida0)
108 print(neuronio,"saida0 =", saida_0)
109 neuronio, saida_1 = teste_generalização(neuronio, padrao_teste_3, saida1)
110 print(neuronio,"saida1 =", saida_1)
111 neuronio, saida_0 = teste_generalização(neuronio, padrao_teste_4, saida0)
112 print(neuronio,"saida0 =", saida_0)
113 neuronio, saida_1 = teste_generalização(neuronio, padrao_teste_5, saida1)
114 print(neuronio,"saida1 =", saida_1)
115
116
117
```

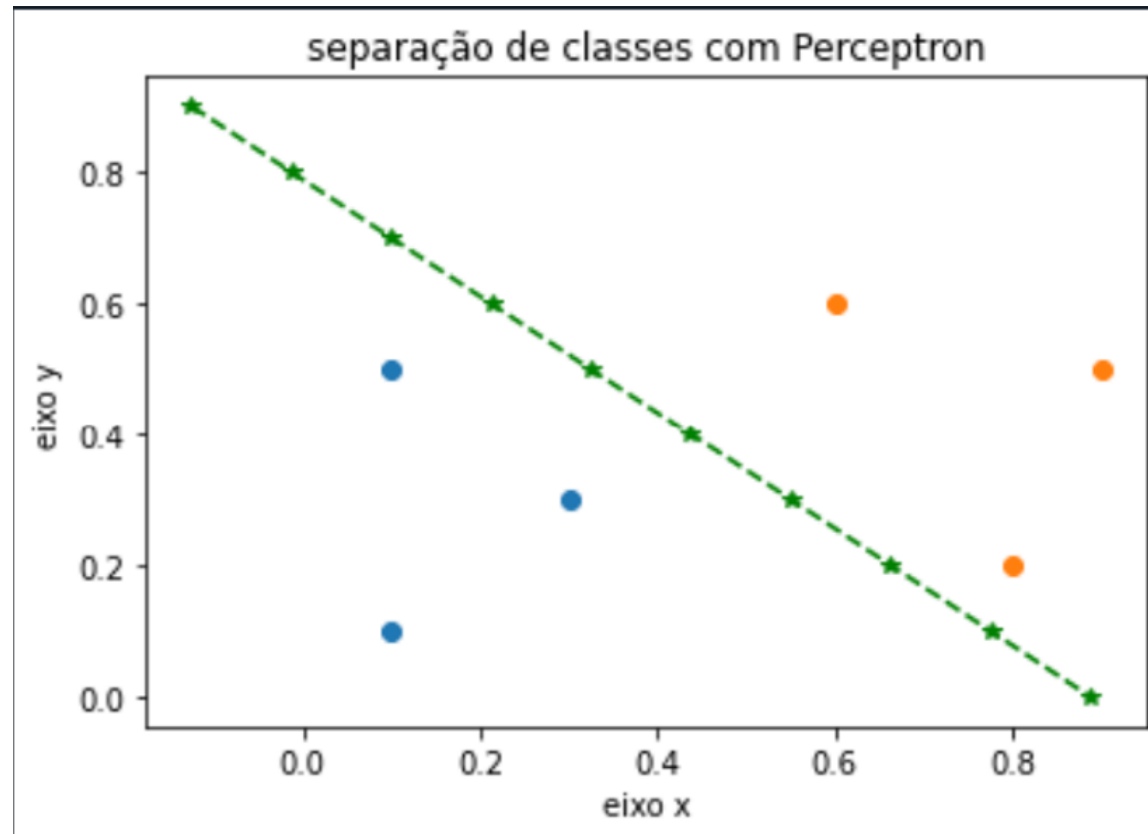


Exercícios

1. Fazer o experimento com neurônio *Perceptron* com os mesmos parâmetros utilizados (pares de entrada $(x1, x2)$ treinados e quantidade de ciclos de treinamento), e os pares para os testes de generalização, verificando os resultados.

```
padrao_0_0 = [-1, 0.1, 0.1]  
padrao_0_1 = [-1, 0.1, 0.5]  
padrao_0_2 = [-1, 0.3, 0.3]  
padrao_1_0 = [-1, 0.6, 0.6]  
padrao_1_1 = [-1, 0.8, 0.2]  
padrao_1_2 = [-1, 0.9, 0.5]
```

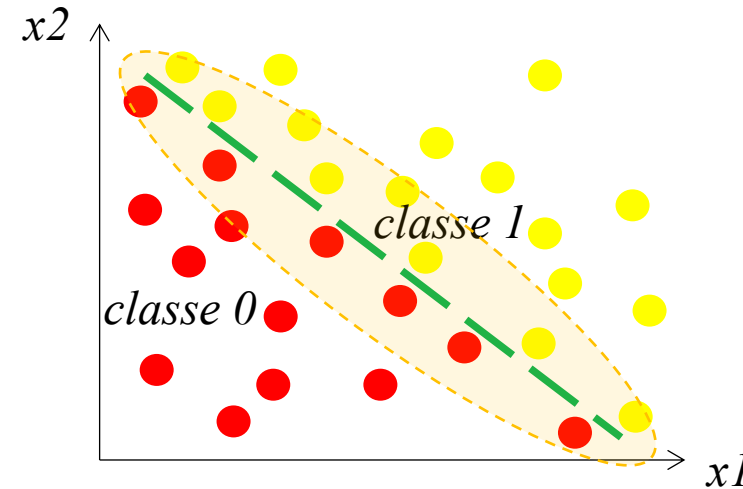
```
#teste de generalização  
padrao_teste_0 = [-1, 0.2, 0.4]  
padrao_teste_1 = [-1, 0.7, 0.8]  
padrao_teste_2 = [-1, 0.6, 0.3]  
padrao_teste_3 = [-1, 0.1, 0.9]  
padrao_teste_4 = [-1, 0.2, 0.6]  
padrao_teste_5 = [-1, 0.8, 0.1]
```



Exercícios

2. Inserir mais pares de treinamento, principalmente na região de fronteira que separa as classes:

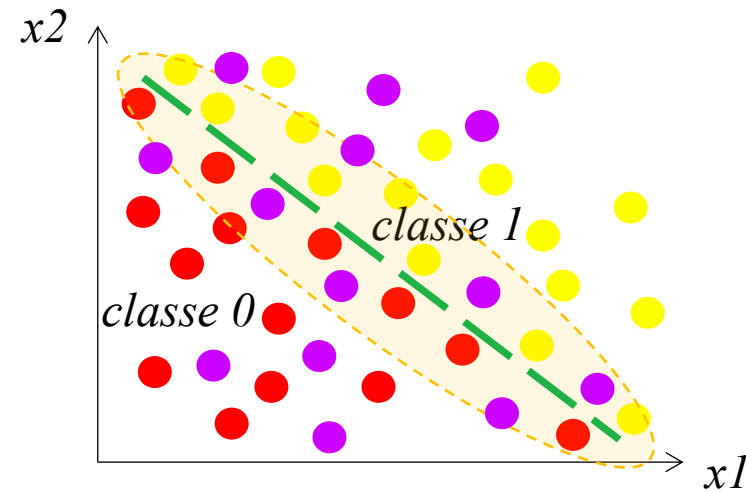
● ● → pares de entrada das classes 0 e 1



Exercícios

3. Repetir o teste de generalização inserindo mais pontos, agora com mais pares já treinados, verificando os resultados. Justificar.

- ● → *pares de entrada das classes 0 e 1*
- → *pares de entrada do teste de generalização*





Matplotlib: Visualização com Python

Matplotlib é uma biblioteca abrangente para a criação de visualizações estáticas, animadas e interativas em Python.



Matplotlib torna as coisas fáceis fáceis e as difíceis possíveis.

Crio

- Desenvolva **gráficos de qualidade de publicação** com apenas algumas linhas de código
- Use **figuras interativas** que podem dar zoom, panorâmica, atualizar ...

Customizar

- Assuma o controle total** dos estilos de linha, propriedades da fonte, propriedades dos eixos ...
- Exporte e incorpore** a vários formatos de arquivo e ambientes interativos

Ampliar

- Explore a funcionalidade personalizada fornecida por **pacotes de terceiros**
- Saiba mais sobre Matplotlib por meio de muitos **recursos de aprendizagem externos**

Documentação

Para começar, leia o [Guia do usuário](#) .

Versão 3.3.3 **mais recente** :
[docs](#) | [changelog](#)

Última versão do Python 2
2.2.5: [docs](#) | [changelog](#)

[Documentos da versão de desenvolvimento](#)

Suporte Matplotlib

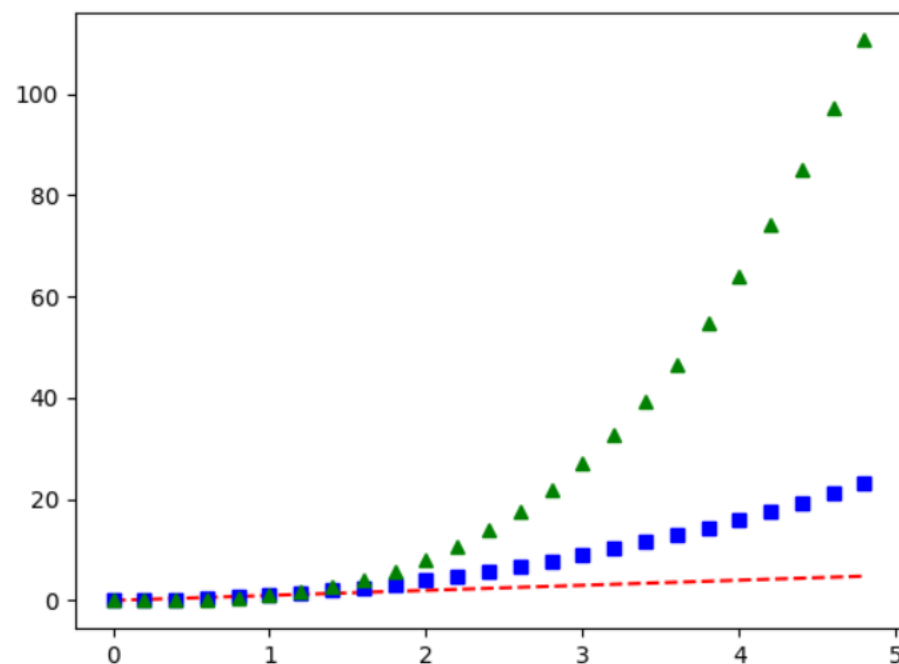
de formato diferentes em uma chamada de função usando matrizes.

```
import numpy as np

# evenly sampled time at 200ms intervals
t = np.arange(0., 5., 0.2)

# red dashes, blue squares and green triangles
plt.plot(t, t, 'r--', t, t**2, 'bs', t, t**3, 'g^')
plt.show()
```

<https://matplotlib.org/tutorials/introductory/pyplot.html#sphx-glr-tutorials-introductory-pyplot-py>



Plotando com strings de palavras-chave

Referências Bibliográficas

- Braga AP, Carvalho APLF, Ludermir TB. *Redes Neurais Artificiais: teoria e aplicações*. Livros Técnicos e Científicos, Rio de Janeiro – RJ; 2007.
- Haykin S. *Neural Networks – A Comprehensive Foundation*. Prentice-Hall; 1994.
- Haykin S. *Redes Neurais – Princípios e prática*. 2a ed.. Porto Alegre: Bookman; 2001.
- Hebb DO. *The Organization of Behavior*. John Wiley & Sons; 1949.
- Heckerman D. *Probabilistic Similarity Networks*. MIT Press, Cambridge, Massachussets; 1991.
- Hopfield JJ. *Neurons with graded response have collective computational properties like those of two-state neurons*. Proceedings of the National Academy of Sciences of the United States of America, 79, 2554-2558; 1982.

Referências Bibliográficas

- Mario MC. *Proposta de Aplicação das Redes Neurais Artificiais Paraconsistentes como Classificador de Sinais Utilizando Aproximação Funcional*. Univ. Federal de Uberlândia, Dissertação de Mestrado, Uberlândia; 2003.
- McCarthy J. *Programs with common sense*. In Proceedings of the Symposium on Mechanisation of Thought Processes, Vol. 1, pp. 77-84, London. Her Majesty's Stationery Office; 1958.
- McCulloch W, Pitts W. *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics 5, 115-133; 1943.
- Rosenblatt F. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms*. Spartan, Chicago; 1962.
- Rumelhart DE, McClelland JL. *Parallel Distributed Processing*. MIT Press, Cambridge, Massachusetts; 1986.
- Turing A. *Computing machinery and intelligence*. Mind, 59, 433-460; 1950.